

Nền tảng phân tích dữ liệu giao thông nhận thức dựa trên kiến trúc Lakehouse và Machine Learning

Nhóm 09

Hà Đăng Long, Nguyễn Hoàng Phúc, Ngô Mạnh Tiến

Tóm tắt nội dung—Bài báo trình bày *Cognitive Traffic Analytics Platform*, một bản mẫu sản phẩm dữ liệu cho phân tích giao thông đô thị Việt Nam dựa trên kiến trúc big data lakehouse và mô hình dự báo tốc độ. Trọng tâm của bài không phải thuật toán mới mà là cách đóng gói pipeline dữ liệu, benchmark ML, API, dashboard và cảnh báo thành một MVP có thể demo cho người vận hành giao thông. Phần thực nghiệm đã kiểm chứng được giới hạn ở benchmark offline trên Kaggle Training Pack 30 ngày, gồm 20 đoạn đường tại Hà Nội và TP.HCM, 57,600 bản ghi ban đầu, 67 cột dữ liệu và 62 feature sau bước chống rò rỉ dữ liệu. Target dự báo được tạo theo từng segment_id cho các horizon 15, 60 và 240 phút; dữ liệu được chia theo thứ tự thời gian để mô phỏng kịch bản huấn luyện trên quá khứ và đánh giá trên tương lai. Trên tập test offline, LightGBM đạt MAE khoảng 1.22–1.30 km/h cho ba horizon và được chọn vì cân bằng giữa sai số, độ trễ suy luận, kích thước artifact, khả năng tích hợp FastAPI và giải thích bằng SHAP. Bài báo phân tách rõ ba mức bằng chứng: benchmark offline đã chạy; pipeline/lakehouse/serving mới ở mức thiết kế, khởi tạo hoặc demo cục bộ; và các hạng mục chưa đủ bằng chứng để tuyên bố production-ready.

Index Terms—Big Data Engineering, traffic forecasting, data lakehouse, machine learning engineering, LightGBM, chronological split, explainable AI.

I. GIỚI THIỆU

A. Bối cảnh

Giao thông đô thị là một bài toán dữ liệu phức tạp vì trạng thái của từng tuyến đường thay đổi liên tục theo giờ cao điểm, ngày trong tuần, mưa lớn, tai nạn, công trình, sự kiện đông người và đặc điểm hạ tầng. Với các thành phố lớn như Hà Nội và Thành phố Hồ Chí Minh, việc chỉ quan sát tốc độ hiện tại chưa đủ để hỗ trợ vận hành: hệ thống cần biết đoạn đường nào có nguy cơ ùn tắc trong tương lai gần, yếu tố nào gây ảnh hưởng và cảnh báo nào đáng ưu tiên.

Các nền tảng phân tích giao thông hiện đại thường cần kết hợp hai nhóm năng lực. Nhóm thứ nhất là *kỹ thuật dữ liệu*: ingestion, streaming, batch processing, quality checks, schema management và lưu trữ dữ liệu lịch sử. Nhóm thứ hai là *kỹ thuật AI*: feature engineering, đánh giá benchmark mô hình, triển khai suy luận, giải thích dự báo và theo dõi chất lượng mô hình. Dự án này đặt mục tiêu nối hai nhóm năng lực đó thành một bản mẫu đầu-cuối cho bối cảnh Việt Nam.

B. Mục tiêu và phạm vi

Mục tiêu của hệ thống là hình thành một MVP sản phẩm phân tích giao thông có thể nối dữ liệu thời gian gần thực, dữ liệu lịch sử, feature store dạng lakehouse, mô hình dự báo tốc độ và dashboard vận hành. Thiết kế hướng tới chuỗi xử lý từ thu thập dữ liệu giao thông/thời tiết/sự kiện, chuẩn hóa theo các lớp dữ liệu, lưu trữ trên S3–Iceberg, huấn luyện mô hình theo road segment và phục vụ kết quả qua FastAPI, bản đồ, dự báo và cảnh báo.

Phạm vi hiện tại là bản mẫu nghiên cứu và demo môn học. Benchmark mô hình đã được chạy trên Kaggle Training Pack; một số thành phần lakehouse/serving đã được thiết kế hoặc khởi tạo; các hạng mục như TomTom Stats loader hoàn chỉnh, OSM importer ổn định, data-quality DAG tự động, online monitoring và retraining vẫn cần bằng chứng chạy thật. Vì vậy, báo cáo sử dụng ngôn ngữ phân tầng bằng chứng thay vì mặc định xem toàn bộ hệ thống là production-ready.

C. Đóng góp chính

Bài báo có bốn đóng góp chính theo hướng sản phẩm dữ liệu:

- 1) Đề xuất kiến trúc lakehouse cho MVP traffic intelligence tại Việt Nam, kết hợp Kafka, Spark, S3, Iceberg, Trino, Airflow, Redis, Neo4j và FastAPI thành một luồng sản phẩm từ dữ liệu đến dashboard.
- 2) Chuẩn hóa pipeline dữ liệu theo ba tầng Bronze/Silver/Gold, có kiểm tra schema, loại trùng, chuẩn hóa thời gian, phát hiện ngoại lệ, lineage và giám sát chất lượng dữ liệu.
- 3) Xây dựng benchmark machine learning cho dự báo tốc độ giao thông 15/60/240 phút, có chronological split, leakage guard và baseline để chứng minh giá trị dự báo trước khi đóng gói thành API.
- 4) Đặc tả trải nghiệm sản phẩm gồm Live Map, forecast page, alert queue, hotspot view, SHAP explanation và monitoring, kèm acceptance gates để tách rõ phần đã kiểm chứng, phần demo và phần cần validation.

II. CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN

A. Dự báo giao thông trong tính toán đô thị

Bài toán dự báo giao thông thường được mô hình hóa như một bài toán chuỗi thời gian hoặc dự báo không gian–thời gian [6]. Ở mức cơ bản, mô hình dựa trên lịch sử tốc độ của chính đoạn đường, thống kê trượt và mẫu lặp theo

giờ trong ngày. Ở mức phức tạp hơn, hệ thống cần kết hợp ngữ cảnh thời tiết, sự kiện, thuộc tính hạ tầng và quan hệ không gian giữa các đoạn đường trong mạng đường.

Dự báo giao thông có hai yêu cầu kỹ thuật song song. Thứ nhất là năng lực mô hình hóa, tức dự báo đủ chính xác theo nhiều horizon. Thứ hai là năng lực kỹ thuật dữ liệu, tức đưa dữ liệu từ ingestion đến lưu trữ, feature, model artifact, API và dashboard một cách có kiểm soát. Bài báo này tập trung vào đóng góp thứ hai: thiết kế nền tảng dữ liệu lớn + AI đầu-cuối, trong đó mô hình LightGBM đóng vai trò baseline triển khai mạnh và dễ kiểm chứng, không phải một thuật toán mới.

B. Các mô hình spatio-temporal graph neural network

Các nghiên cứu hiện đại thường biểu diễn mạng giao thông như đồ thị, trong đó node là cảm biến/đoạn đường và cạnh thể hiện quan hệ không gian hoặc phụ thuộc học được từ dữ liệu. DCRNN mô hình hóa giao thông như quá trình khuếch tán trên đồ thị có hướng và kết hợp graph diffusion với encoder-decoder recurrent network [7]. STGCN thay recurrent unit bằng khối convolution không gian-thời gian để học phụ thuộc không gian và thời gian hiệu quả hơn [8]. Graph WaveNet bổ sung adaptive dependency matrix và dilated causal convolution để học quan hệ ẩn và chuỗi dài [9]. T-GCN kết hợp graph convolutional network với gated recurrent unit nhằm học đồng thời phụ thuộc không gian và biến động thời gian [10].

Các hướng trên rất phù hợp khi có dữ liệu cảm biến lớn, đồ thị đường ổn định, chuỗi lịch sử dài và quy trình huấn luyện/đánh giá đủ chặt. Trong phạm vi hiện tại, dataset chỉ có 30 ngày, 20 đoạn đường và 2 thành phố; pipeline online chưa có bằng chứng chạy end-to-end, và mục tiêu của dự án là bản mẫu production-oriented có inference nhanh, artifact nhỏ, dễ tích hợp FastAPI và có giải thích SHAP. Vì vậy, deep learning/graph neural network được xem là hướng mở rộng sau khi có thêm dữ liệu, ground-truth và graph mapping ổn định, thay vì lựa chọn chính ở giai đoạn này.

C. Data lakehouse cho dữ liệu giao thông

Dữ liệu giao thông có đặc điểm vừa streaming vừa batch: trạng thái hiện tại cần cập nhật liên tục, trong khi dữ liệu lịch sử cần được lưu để phân tích, huấn luyện và kiểm toán. Kiến trúc lakehouse phù hợp với yêu cầu này vì kết hợp object storage chi phí thấp với table format hỗ trợ schema evolution, partition pruning và snapshot. Trong dự án, Amazon S3 giữ vai trò object storage, Apache Iceberg giữ vai trò table format, Hive Metastore quản lý metadata và Trino phục vụ truy vấn SQL [3]. Kafka được dùng cho event streaming [2], còn Spark xử lý streaming/batch trước khi ghi vào các lớp Bronze/Silver/Gold [1].

D. Mô hình dạng bảng machine learning cho dự báo

Các mô hình gradient boosting trên cây quyết định như LightGBM, XGBoost và CatBoost thường mạnh trên dữ

liệu dạng bảng có nhiều feature thủ công [4], [5], [11]. Với traffic forecasting ở dạng tabular feature, nhóm mô hình này có ưu điểm là huấn luyện nhanh, suy luận nhanh, artifact dễ đóng gói và giải thích được bằng feature importance hoặc SHAP [12]. Do đó, chúng phù hợp với giai đoạn bản mẫu kỹ thuật AI, nơi yêu cầu không chỉ là điểm số offline mà còn là khả năng đóng gói artifact, phục vụ API, kiểm soát schema và vận hành dashboard.

E. Khoảng trống mà dự án tập trung xử lý

Nhiều demo dự báo chỉ dừng ở notebook, trong khi nhiều demo dashboard chỉ dùng dữ liệu giả hoặc API giả lập. Dự án này nối hai phía: pipeline dữ liệu có cấu trúc, lưu trữ theo lakehouse, tạo feature phục vụ model, đóng gói artifact và thiết kế serving layer. Vì vậy, đóng góp chính nằm ở hướng *nền tảng dữ liệu AI đầu-cuối cho traffic analytics*, không phải tối ưu một thuật toán đơn lẻ.

F. Định nghĩa bài toán và kiểm soát leakage

Với mỗi đoạn đường s và thời điểm rời rạc t theo bucket 15 phút, gọi $v_{s,t}$ là tốc độ quan sát và $\mathcal{F}_{s,t}$ là tập feature chỉ được phép dùng thông tin có timestamp không lớn hơn t . Bài toán dự báo horizon h được viết dưới dạng

$$\hat{y}_{s,t}^{(h)} = f_h(\mathcal{F}_{s,t}), \quad y_{s,t}^{(h)} = v_{s,t+h}, \quad h \in \{1, 4, 16\}. \quad (1)$$

Điều kiện chống rò rỉ là $\mathcal{F}_{s,t} \cap \{v_{s,t+1}, \dots, v_{s,t+h}\} = \emptyset$ và mọi rolling/window feature phải được tính từ quá khứ hoặc hiện tại. Chronological split được ký hiệu $T_{train} < T_{val} < T_{test}$, bảo đảm toàn bộ quan sát trong test xảy ra sau train/validation. Công thức này làm rõ mục tiêu benchmark: ước lượng năng lực dự báo tương lai cho sản phẩm cảnh báo, không phải khớp lại pattern đã nhìn thấy trong dữ liệu tương lai.

III. KIẾN TRÚC HỆ THỐNG

A. Tổng quan kiến trúc

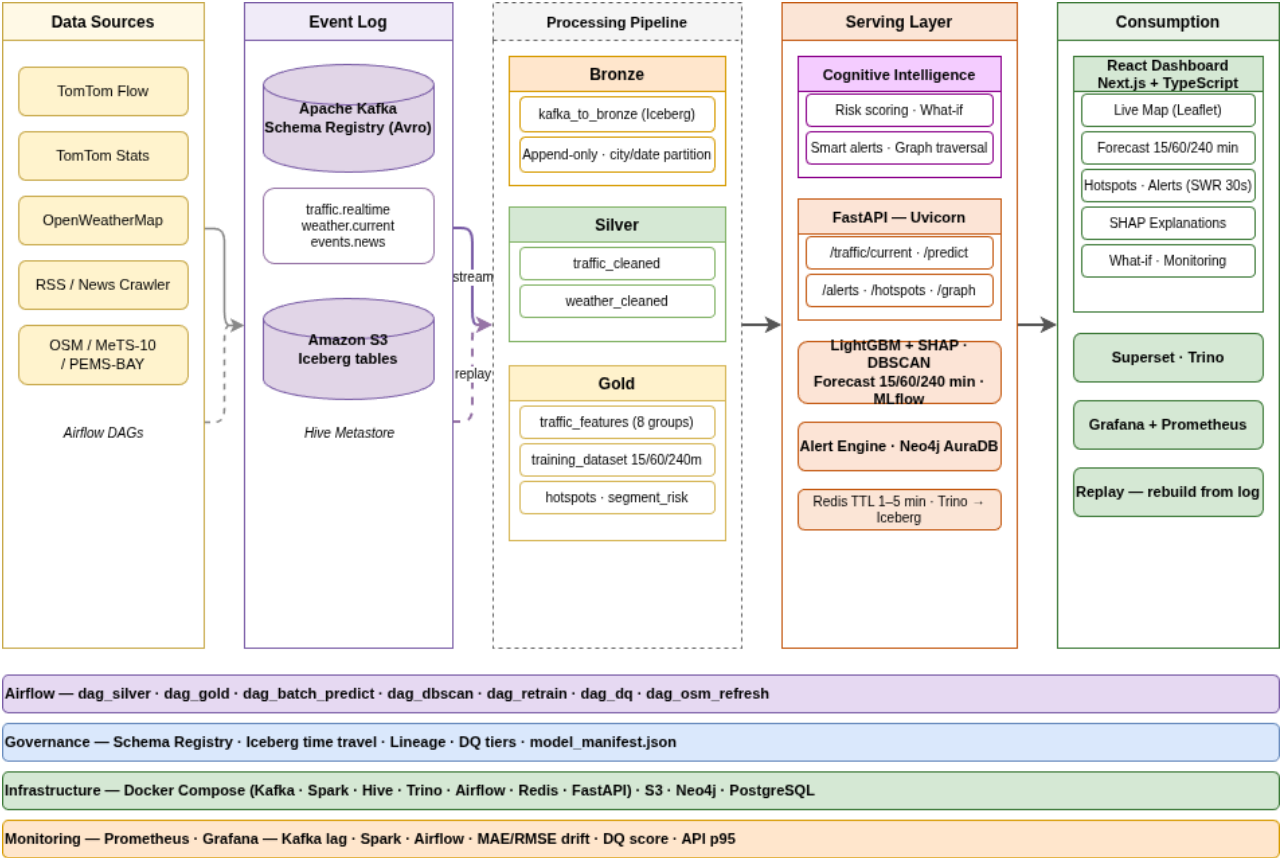
Hình 1 mô tả kiến trúc tổng thể của nền tảng. Hệ thống được chia thành năm lớp: nguồn dữ liệu, ingestion/orchestration, lakehouse processing, analytics/modeling và serving/visualization.

B. Tech stack

Bảng I tóm tắt các thành phần chính. Điểm quan trọng không nằm ở việc liệt kê nhiều công nghệ, mà ở cách tách vai trò: Kafka nhận message, Spark chuẩn hóa dữ liệu, S3-Iceberg lưu trữ có version, Trino truy vấn lakehouse, Airflow điều phối batch/DQ, FastAPI phục vụ suy luận, Redis giảm độ trễ đọc gần nhất và Neo4j hỗ trợ phân tích mạng đường.

C. Luồng dữ liệu đầu-cuối

Luồng dữ liệu mục tiêu được thiết kế theo chuỗi $\text{Kafka} \rightarrow \text{Spark} \rightarrow \text{Bronze/Silver/Gold} \rightarrow \text{Iceberg/Trino} \rightarrow \text{FastAPI/Dashboard}$. Ở bước ingestion, producer ghi message vào các Kafka topics theo từng nguồn dữ liệu và đính kèm metadata như `source`, `event_time`,



Hình 1. Kiến trúc tổng quan của Cognitive Traffic Analytics Platform.

Bảng I
VAI TRÒ KIẾN TRÚC CỦA CÁC THÀNH PHẦN CÔNG NGHỆ CHÍNH

Lớp	Công nghệ	Vai trò
Thu thập dữ liệu	Python Producer, Kafka, Schema Registry	Polling dữ liệu giao thông/thời tiết/tin tức, kiểm soát schema, retry và dead-letter queue
Xử lý Storage	Spark Structured Streaming, PySpark Amazon S3, Apache Iceberg	Đọc Kafka, xử lý streaming/batch, xây dựng ba tầng dữ liệu Lưu Parquet, schema evolution, partitioning, snapshot/time travel
Catalog/truy vấn Orchestration Phục vụ	Hive Metastore, PostgreSQL, Trino Apache Airflow FastAPI, Redis	Quản lý metadata và truy vấn Iceberg tables Lên lịch batch ingestion, DQ, feature engineering, retraining API dự báo/cảnh báo, cache trạng thái thời gian gần thực và kết quả dự báo
Đồ thị Học máy Trực quan hóa	Neo4j, NetworkX, OSMnx LightGBM, scikit-learn, SHAP, MLflow Next.js, Leaflet, Recharts, Superset, Grafana	Lưu mạng đường, centrality, upstream/downstream analysis Dự báo, phân cụm, giải thích mô hình và vòng đời mô hình Dashboard, bản đồ, báo cáo BI và giám sát

`ingestion_time`, `schema_version` và `pipeline_run_id`. Spark Structured Streaming đọc topic, kiểm tra schema contract, ghi bản ghi hợp lệ vào Bronze và chuyển bản ghi lỗi sang dead-letter queue. Silver chuẩn hóa timestamp, múi giờ, đơn vị đo, khóa `segment_id`, duplicate và outlier. Gold sinh feature, target, prediction, alert và dashboard aggregate. Trino truy vấn các Iceberg tables cho phân tích ad-hoc, còn FastAPI dùng Gold/model artifact để phục vụ dashboard.

Điểm quan trọng là bài báo không coi toàn bộ chuỗi trên là đã vận hành production. Benchmark offline và

model artifact đã có bằng chứng từ notebook; các thành phần Kafka/Spark/Iceberg/Airflow/FastAPI được trình bày theo mức bằng chứng tương ứng, không mặc định là đã vận hành đầy đủ.

D. Schema contract, partition, checkpoint và replay

Schema contract tối thiểu cho traffic event gồm `segment_id`, `city`, `event_time`, `speed_kmh`, `freelflow_speed_kmh`, `jam_factor`, `source`, `schema_version` và `ingestion_time`. Weather/event payload được tách topic riêng nhưng phải có khóa thời

gian và địa lý để join theo time bucket. Breaking schema change phải bị chặn ở Schema Registry hoặc đẩy sang `pipeline.dlq`, không được silently cast trong Spark job.

Partition strategy của Iceberg ưu tiên truy vấn theo thành phố và thời gian: `city/year/month/day`. Với bảng prediction hoặc alert, có thể bổ sung partition theo horizon hoặc severity nếu truy vấn dashboard chứng minh cần thiết. Spark checkpoint được lưu dưới prefix `checkpoints/` tách theo job và topic để đảm bảo exactly-once/at-least-once semantics tùy sink. Bronze giữ payload gần nguồn để phục vụ replay khi logic Silver/Gold thay đổi. Lineage được duy trì bằng các cột `_source`, `_ingested_at`, `_pipeline_run_id`, `_schema_version` và `_quality_flags`. Monitoring metrics cần có gồm Kafka consumer lag, Spark batch duration, input/output row count, DLQ rate, Iceberg commit latency, Trino query latency, API p95 latency, cache hit rate, data freshness và model drift.

E. Lakehouse trên Amazon S3

Object storage được chọn là Amazon S3 với các prefix `bronze/`, `silver/`, `gold/`, `checkpoints/`, `models/` và `reports/`. Spark và Trino truy cập dữ liệu qua giao thức `s3a://`; Iceberg và Hive Metastore đảm nhiệm metadata, schema và snapshot. Đối với môi trường demo, lifecycle rule được khuyến nghị để xóa dữ liệu tạm sau 7–14 ngày, giúp kiểm soát chi phí. Credentials được cấu hình qua `.env`, không hardcode trong code hoặc notebook.

F. Trạng thái triển khai và bằng chứng

Để tránh đánh đồng giữa kết quả notebook, bản mẫu hệ thống và production system, bài báo sử dụng ba mức bằng chứng xuyên suốt. Mức thứ nhất là *benchmark verified*: dataset, notebook, metric test và model artifact đã có thể đối chiếu. Mức thứ hai là *demo/design evidence*: các thành phần như S3 layout, FastAPI, Redis hoặc dashboard được mô tả theo thiết kế và chỉ được xem là đã kiểm chứng khi có log, response mẫu hoặc ảnh chụp demo đi kèm. Mức thứ ba là *future validation*: các phần như TomTom Stats loader, OSM importer ổn định, Airflow DQ DAG, online monitoring, load test và ground-truth cho hotspot/alert chưa được xem là kết quả hoàn chỉnh. Cách trình bày này giữ báo cáo trung thực: đóng góp đã chắc nằm ở benchmark offline và thiết kế nền tảng; phần còn lại là lộ trình kiểm chứng kỹ thuật.

G. Kafka topics và schema contract

Kafka được dùng làm xương sống truyền message cho dữ liệu thời gian gần thực. Mỗi producer ghi vào topic riêng để giảm phụ thuộc trực tiếp giữa nguồn dữ liệu và consumer: `traffic.realtime` cho trạng thái giao thông, `weather.current` cho thời tiết, `events.news` cho tin tức/sự kiện đã xử lý, `model.predictions` cho kết quả dự báo, `traffic.alerts` cho cảnh báo và `pipeline.dlq` cho message lỗi schema hoặc lỗi xử lý. Schema Registry giúp kiểm soát cấu trúc message, tránh trường hợp producer

thay đổi trường làm Spark job hoặc API downstream bị lỗi ngầm.

H. Thiết kế Iceberg tables

Các Iceberg tables được partition theo `city/year/month/day` để tối ưu truy vấn theo thời gian và khu vực. Các bảng dữ liệu gốc giữ payload thô cùng metadata ingestion. Các bảng dữ liệu đã chuẩn hóa kiểm tra schema, múi giờ, trùng lặp và ngoại lệ. Các bảng phục vụ phân tích/mô hình được thiết kế theo mục tiêu sử dụng: training dataset, prediction result, alert, hotspot và dashboard aggregate.

IV. PIPELINE DỮ LIỆU

A. Nguồn dữ liệu và khả năng tái lập

Thực nghiệm trong bài báo sử dụng nguồn dữ liệu và notebook công khai trên Kaggle. Bộ dữ liệu chính là *Vietnamese Kaggle Dataset*, chứa training pack 30 ngày dùng cho EDA, feature validation và benchmark mô hình. Notebook huấn luyện đi kèm được dùng làm bằng chứng tái lập cho các bước tạo target, chống leakage, chronological split, benchmark nhiều mô hình và đóng gói artifact.

Nguồn tái lập chính gồm dataset Kaggle và notebook huấn luyện công khai [14], [15]. Dataset cung cấp training pack 30 ngày cho Hà Nội/TP.HCM; notebook ghi lại EDA, target generation, leakage guard, chronological split, benchmark nhiều mô hình và đóng gói artifact.

Các nguồn như TomTom, OpenWeatherMap, OpenStreetMap và news/RSS được xem là hướng tích hợp cho pipeline thu thập dữ liệu thực tế. Trong phạm vi benchmark của bài báo, kết quả mô hình được báo cáo dựa trên Kaggle training pack để đảm bảo tính tái lập.

B. Mô hình ba tầng dữ liệu

Thay vì xem ba tầng dữ liệu như tên gọi hình thức, bài báo diễn giải pipeline theo vai trò dữ liệu. Tầng thứ nhất là *vùng dữ liệu gốc*, nơi giữ record gần với nguồn ban đầu để truy vết và replay. Tầng thứ hai là *vùng dữ liệu đã chuẩn hóa*, nơi dữ liệu được làm sạch, đồng bộ timestamp, loại trùng lặp/ngoại lệ và join các ngữ cảnh cần thiết. Tầng thứ ba là *vùng dữ liệu phục vụ phân tích và mô hình*, nơi các feature, target, prediction, alert và dashboard artifacts được sinh ra.

C. Chất lượng dữ liệu và chống leakage

Trong traffic forecasting, lỗi dữ liệu có thể làm mô hình đạt điểm offline cao nhưng thất bại khi triển khai. Pipeline vì vậy kiểm soát ba nhóm rủi ro: lỗi dữ liệu đầu vào, lỗi tích hợp giữa nguồn và rò rỉ dữ liệu. Ở mức đầu vào, các khóa `segment_id`, timestamp, tốc độ và city phải hợp lệ; tốc độ cần nằm trong miền vật lý hợp lý, ví dụ 0–150 km/h, và `jamFactor` nằm trong khoảng [0, 10]. Ở mức tích hợp, traffic, weather và event phải dùng cùng time bucket 15 phút, có cờ missing/imputed thay vì silent

Bảng II
MA TRẬN BẢNG CHỨNG THEO GÓC NHÌN SẢN PHẨM

Hạng mục sản phẩm	Mức bằng chứng	Artifact nền nẹp	Điều kiện nâng mức
Forecast benchmark	Benchmark verified	Kaggle dataset, notebook, split/metric output	Pin dataset version, export run log, checksum và model artifact hash.
Model serving pack	Benchmark verified	joblib, manifest, feature schema, smoke test	Gắn MLflow registry, model signature, rollback policy và latency test.
Live Map/Forecast UI	Demo/design evidence	API sample response, screenshot dashboard	Chứng minh response thật có timestamp, stale badge, schema version và forecast horizon.
Lakehouse layout	Demo/design evidence	S3 prefix, Iceberg DDL, sample Trino query	Chứng minh Spark ghi Iceberg, snapshot/time travel và data freshness report.
Alert/Hotspot workflow	Future validation	Rule/DBSCAN design, sample output	Cần ground truth, precision/recall/F1, calibration và false-alert review.
Ops monitoring	Future validation	Kafka/Spark/Airflow/Grafana logs	Cần E2E run, API p95, DLQ rate, cache hit rate, drift report và load test.

drop. Ở mức leakage guard, mọi cột future-like, target-like, prediction-like hoặc rolling window nhìn về tương lai phải bị loại trước khi huấn luyện; nếu schema input lệch với `feature_schema_used.json`, training hoặc inference cần dừng để yêu cầu migration/versioning.

D. Kỹ thuật feature

Lớp dữ liệu phục vụ phân tích/ML tạo feature theo tám nhóm chính. Nhóm thời gian mô tả `hour_of_day`, `day_of_week`, cuối tuần và giờ cao điểm. Nhóm trạng thái giao thông mô tả tốc độ hiện tại, tốc độ tự do, `jamFactor` và tỷ lệ ùn tắc. Nhóm lag/rolling biểu diễn quán tính giao thông qua speed lag, jam lag và rolling mean. Nhóm thời tiết gồm mưa, độ ẩm, gió và tầm nhìn. Nhóm không gian và POI phân biệt city, district, road class, segment length và mật độ điểm quan tâm. Nhóm sự kiện mô tả accident, flood, roadwork hoặc event score. Nhóm baseline thống kê so sánh tốc độ hiện tại với p15/p50/p85 lịch sử theo cùng khung giờ. Nhóm đồ thị mô tả vai trò của segment bằng degree, betweenness hoặc upstream/downstream congestion. Các nhóm này được giữ ở mức tabular feature để phù hợp với LightGBM và dễ kiểm soát feature parity giữa offline training và online inference.

E. Tạo target theo horizon

Target được tạo độc lập theo từng `segment_id` để tránh trộn tín hiệu giữa các tuyến đường. Sau khi sắp xếp theo `segment_id` và `timestamp`, notebook dịch `current_speed` về tương lai bằng group shift: horizon 15 phút tương ứng 1 bước, 60 phút tương ứng 4 bước và 240 phút tương ứng 16 bước trên dữ liệu 15 phút. Các dòng cuối mỗi segment không còn đủ quan sát tương lai phải bị loại khỏi tập huấn luyện cho horizon tương ứng. Vì vậy, số dòng dùng để train/evaluate theo từng horizon cần được ghi nhận sau bước drop target-null, không chỉ dựa trên số dòng raw ban đầu.

V. THIẾT LẬP THỰC NGHIỆM

A. Dữ liệu, target và chia tách

Notebook huấn luyện sử dụng `gold_training_dataset.parquet` từ Kaggle Training Pack 30 ngày [14], [15]. Dataset ban đầu có 57,600

Bảng III
TÓM TẮT DỮ LIỆU, FEATURE SCHEMA VÀ SPLIT BENCHMARK

Thuộc tính	Giá trị
Số bản ghi ban đầu	57,600
Số cột ban đầu	67
Số segment / thành phố	20 / 2
Khoảng thời gian	2026-05-10–2026-06-08
Tần suất quan sát	15 phút
Horizon dự báo	15m, 60m, 240m
Feature số / phân loại	54 / 8
Tổng feature sau leakage guard	62
Cột nghi leakage sau guard	0
Train / validation / test	40,060 / 8,660 / 8,560
Tỷ lệ split xấp xỉ	69.9 / 15.1 / 14.9%

bản ghi và 67 cột, tương ứng 20 road segments tại Hà Nội và TP.HCM trong khoảng 2026-05-10 00:00:00 đến 2026-06-08 23:45:00. Mỗi bản ghi biểu diễn trạng thái segment theo time bucket 15 phút, kèm các nhóm feature về traffic hiện tại, lịch sử gần, thời tiết, ngữ cảnh đô thị, sự kiện/POI và đặc trưng đồ thị nếu có.

Ba target được định nghĩa là tốc độ tương lai của chính segment: `future_speed_15m`, `future_speed_60m` và `future_speed_240m`. Nếu target chưa có sẵn trong dataset, notebook tạo target bằng group shift theo `segment_id`; nếu target đã có sẵn, notebook vẫn kiểm tra rằng target không xuất hiện trong feature set. Sau leakage guard, các cột future-like, target-like, prediction-like, raw timestamp và metadata không phù hợp bị loại khỏi input mô hình. Feature set cuối gồm 54 feature số và 8 feature phân loại, tổng cộng 62 feature dùng cho huấn luyện.

Vì traffic forecasting phụ thuộc mạnh vào thứ tự thời gian, notebook sử dụng chronological split thay vì random split. Random split có thể đưa quan sát tương lai của cùng đoạn đường vào train trong khi quan sát quá khứ nằm ở test, làm metric lạc quan. Chronological split giữ đúng giả lập triển khai: mô hình được huấn luyện trên quá khứ, chọn tham số trên validation kế tiếp và chỉ báo cáo kết quả cuối trên test sau cùng. Bảng III tóm tắt dữ liệu, schema feature và split hậu xử lý được đối chiếu từ notebook.

B. Thiết kế thí nghiệm chống quá khớp

Thí nghiệm được thiết kế theo hướng production-oriented. Tập validation dùng để lựa chọn mô hình và

tham số, trong khi tập test chỉ dùng cho báo cáo cuối. Các baseline đơn giản như train mean, current speed, speed lag và historical p50 được giữ lại để kiểm tra xem mô hình phức tạp có thật sự tạo thêm giá trị hay không. Ngoài metric tổng thể, notebook còn xuất slice metrics theo thành phố, loại đường, time band và condition để phát hiện trường hợp mô hình tốt trung bình nhưng kém ở một nhóm cụ thể.

C. Chỉ số đánh giá

Metric chính là Mean Absolute Error (MAE) vì đơn vị trực tiếp là km/h và dễ diễn giải cho người dùng dashboard. RMSE được dùng bổ sung để phạt mạnh các lỗi lớn. Median absolute error, P90 và P95 absolute error giúp đánh giá tail risk, phù hợp với bài toán cảnh báo giao thông vì một vài lỗi lớn ở giờ cao điểm có thể ảnh hưởng trải nghiệm vận hành.

VI. MÔ HÌNH DỰ BÁO TỐC ĐỘ

A. Benchmark models

Notebook đánh giá ba nhóm mô hình. Nhóm baseline gồm Train Mean, Current Speed, Speed Lag/Rolling Mean và Historical p50 để kiểm tra mô hình phức tạp có thật sự vượt các heuristic đơn giản hay không. Nhóm tuyến tính gồm Ridge Regression để đo mức học được từ quan hệ gần tuyến tính. Nhóm tree/boosting gồm Extra Trees, HistGradientBoosting, XGBoost [11], CatBoost [5] và LightGBM [4]; nhóm này phù hợp với dữ liệu tabular có feature traffic, weather, temporal, event và spatial đã được thiết kế thủ công.

B. Tiêu chí chọn mô hình

Model selection trong bài báo được định hướng bởi triển khai, không chỉ bởi metric offline. CatBoost có thể đạt MAE thấp hơn ở một số horizon, nhưng LightGBM được chọn nếu sai số nằm trong ngưỡng gần best model vì có các ưu điểm kỹ thuật: inference nhanh, artifact nhỏ, dễ đóng gói bằng joblib, dễ tích hợp FastAPI, SHAP-friendly và phổ biến trong tabular ML production. Do đó, quyết định chọn LightGBM là *production-oriented*, không phải khẳng định LightGBM luôn là mô hình có MAE thấp nhất tuyệt đối.

C. Ablation cần kiểm chứng bổ sung

Ablation study chưa được xem là kết quả thực nghiệm chính vì notebook hiện tại chưa cung cấp đủ metric theo từng nhóm feature. Do đó, bài báo chỉ nêu đây là *required validation artifact*: cần chạy lần lượt các cấu hình “current speed only”, thêm lag features, thêm rolling statistics, thêm weather/context và cuối cùng thêm event/POI/graph features; mỗi cấu hình phải báo cáo MAE/RMSE/P90 theo 15/60/240 phút và slice metric theo city/time band. Không có số liệu ablation ước lượng nào được đưa vào bảng kết quả chính.

Bảng IV
KẾT QUẢ LIGHTGBM TRÊN TẬP TEST CỦA BENCHMARK OFFLINE

Horizon	MAE	P50 AE	P90 AE	P95 AE
15 phút	1.224	0.958	2.582	3.168
60 phút	1.272	0.989	2.625	3.303
240 phút	1.302	1.029	2.729	3.354

Bảng V
SO SÁNH NHANH CÁC MÔ HÌNH TỐT NHẤT TRÊN HORIZON 15 PHÚT

Model	MAE	RMSE	R^2
CatBoost	1.207	1.620	0.954
HistGradientBoosting	1.215	1.630	0.953
XGBoost	1.217	1.633	0.953
LightGBM	1.224	1.641	0.952
Extra Trees	1.244	1.669	0.951
Ridge	1.276	1.708	0.948
Historical p50	1.454	1.947	0.933
Current speed	1.615	2.142	0.919
Train mean	5.805	7.529	-0.002

D. Kết quả đánh giá benchmark

Bảng IV trình bày kết quả LightGBM trên tập test của benchmark offline. Sai số MAE dưới 1.31 km/h cho cả ba horizon cho thấy dataset training pack có pattern dự báo rõ sau bước feature engineering. Tuy nhiên, kết quả này cần được diễn giải thận trọng vì dataset chỉ gồm 30 ngày, 20 đoạn đường và đã được chuẩn hóa; dữ liệu nhỏ, ít nhiễu hoặc có pattern lặp mạnh có thể làm MAE offline thấp hơn kỳ vọng trong vận hành thật. Do đó, kết quả này là bằng chứng benchmark offline, không phải cam kết production; cần online validation khi feature pipeline realtime được chạy đầy đủ.

Bảng V bổ sung một lát cắt ranking ở horizon 15 phút để minh họa lý do chọn LightGBM theo tiêu chí triển khai: CatBoost có MAE thấp hơn rất nhỏ, nhưng chênh lệch này không đủ để loại bỏ các lợi thế về tốc độ, kích thước artifact và tích hợp serving của LightGBM.

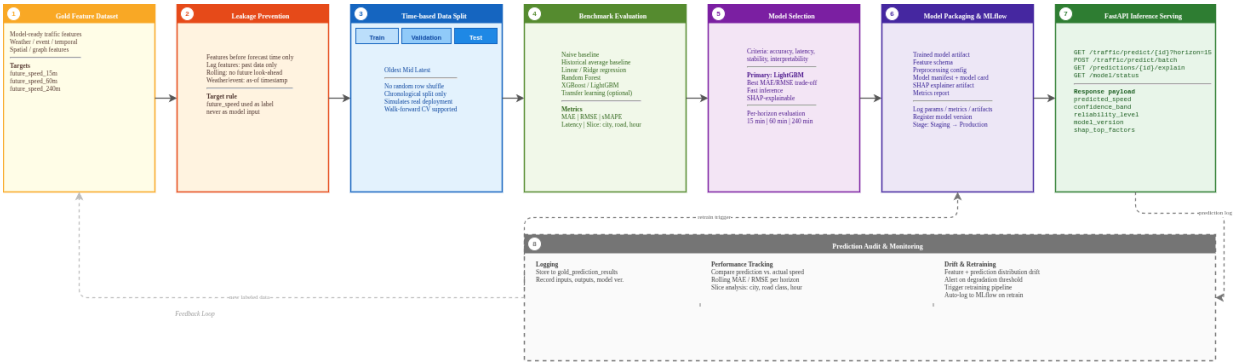
E. Inference artifact và reliability

Notebook xuất model pack gồm `selected_model_<horizon>_lightgbm.joblib`, `model_manifest.json`, `feature_schema_used.json`, đánh giá benchmark bằng xếp hạng, dự báo table, slice metrics, walk-forward metrics và SHAP top feature [12]. Inference smoke test đã chạy thành công cho cả ba horizon. Đây là điểm quan trọng để chứng minh pipeline không dừng ở training notebook mà có artifact phục vụ backend.

VII. SẢN PHẨM, SERVING VÀ CẢNH BÁO

A. Người dùng và luồng giá trị sản phẩm

MVP được thiết kế cho ba nhóm người dùng. Người vận hành giao thông cần một màn hình để biết đoạn nào đang xấu đi, dự báo trong 15–60 phút tới và cảnh báo nào cần xử lý trước. Nhà phân tích dữ liệu cần truy vấn lịch sử, so sánh tốc độ với baseline và kiểm tra nguyên nhân



Hình 2. Vòng đời huấn luyện và triển khai mô hình dự báo: Gold features → leakage guard → chronological split → benchmark → artifact → FastAPI inference.

Bảng VI
CONTRACT ARTIFACT PHỤC VỤ SẢN PHẨM DỰ BÁO

Artifact	Vai trò trong sản phẩm
Model binary	Tải đúng mô hình theo horizon và version.
Manifest	Ghi dataset, split, metric, thư viện và thời điểm train.
Feature schema	Chặn inference khi thiếu/thừa feature hoặc sai kiểu.
Benchmark output	Số sánh model candidate và baseline trước deploy.
SHAP top factors	Giải thích dự báo cho dashboard và debug drift.
Smoke test log	Kiểm tra model load, schema transform và latency cơ bản.

dự báo qua SHAP. Data/ML engineer cần quan sát data freshness, schema compatibility, model version và drift để biết khi nào dữ liệu hoặc mô hình không còn đáng tin cậy.

Luồng sử dụng chính của sản phẩm là: mở Live Map để xem trạng thái mới nhất, chọn segment có rủi ro, xem forecast 15/60/240 phút, đọc các yếu tố giải thích, xác nhận hoặc bỏ qua alert, sau đó kiểm tra monitoring nếu dữ liệu bị stale. Cách mô tả này giúp dashboard không chỉ là giao diện minh họa mà trở thành một workflow vận hành có đầu vào, đầu ra, trạng thái và tiêu chí đo.

B. Lớp phục vụ FastAPI

Serving layer biến Gold/model artifact thành sản phẩm sử dụng được qua dashboard. FastAPI cung cấp các endpoint cho trạng thái traffic hiện tại, dự báo đa horizon, cảnh báo, hotspot, SHAP explanation [12] và monitoring. Redis được dùng theo hai kiểu cache: cache trạng thái realtime có TTL ngắn để bản đồ phản hồi nhanh, và cache dự báo có TTL dài hơn để tránh gọi model lặp lại cho cùng segment_id/horizon. API được tách khỏi training pipeline; khi cần dữ liệu lịch sử, API truy vấn Trino/Iceberg hoặc đọc bảng Gold đã chuẩn bị, còn Neo4j phục vụ truy vấn upstream/downstream theo mạng đường.

Các endpoint trọng tâm được gom theo chức năng thay vì trình bày như checklist API: /traffic/current phục vụ trạng thái giao thông mới nhất; /traffic/forecast

Bảng VII
LUỒNG SẢN PHẨM TỐI THIỂU CỦA MVP

Màn hình	Câu hỏi sản phẩm	Dữ liệu/API cần chứng minh
Live Map	Đường nào đang xấu đi?	GeoJSON segment, current speed, congestion level, last updated.
Forecast	15/60/240 phút tới ra sao?	Prediction payload, horizon, baseline, model version.
Alerts	Cảnh báo nào cần ưu tiên?	Severity, rule/model reason, acknowledge status, stale-data guard.
Hotspots	Khu vực nào có cụm rủi ro?	Cluster id, affected segments, centroid, confidence.
Explain	Vì sao dự báo như vậy?	SHAP top factors, feature snapshot, response schema version.
Monitoring	Có tin được dữ liệu không?	Kafka lag, API p95, DLQ rate, freshness, drift indicator.

trả về dự báo theo segment_id và horizon; /segments/geojson cung cấp geometry cho bản đồ; /alerts/active và /hotspots phục vụ vận hành cảnh báo/cụm ùn tắc; /predictions/explain trả về SHAP top factors; còn /monitoring/pipeline dành cho telemetry của pipeline, API và model. Khi triển khai thật, mỗi endpoint cần kèm versioning, schema response, thời điểm cập nhật cuối, trạng thái cache và mã lỗi rõ ràng để dashboard không hiển thị dự báo cũ như dữ liệu mới.

C. Dashboard trực quan

Dashboard nên được mô tả theo góc nhìn sản phẩm thay vì chỉ liệt kê màn hình. Live Map hiển thị GeoJSON/polyline theo từng road segment, tô màu theo congestion hoặc predicted risk thay vì phủ heatmap ngẫu nhiên trên toàn thành phố. Forecast page cho phép chọn segment và hiển thị dự báo 15/60/240 phút kèm baseline lịch sử. Alerts page quản lý hàng đợi cảnh báo với severity, trạng thái acknowledge và thời điểm cập nhật cuối.

Hotspots page hiển thị cụm DBSCAN hoặc predicted risk cluster. Explanations page trình bày SHAP top factors để giải thích vì sao tốc độ được dự báo giảm/tăng. Monitoring page theo dõi Kafka lag, Spark job duration, API p95 latency, data quality và model drift khi telemetry được kết nối.

Về mặt sản phẩm, dashboard nên ưu tiên bốn màn hình có khả năng chứng minh dữ liệu thật: Live Map theo polyline segment, Forecast theo horizon, Alerts/Hotspots theo rủi ro vận hành và Explanations/Monitoring cho khả năng giải thích và quan sát. Các màn hình phụ như Settings hoặc BI report chỉ nên đưa vào demo khi đã có API và telemetry tương ứng, tránh tạo cảm giác giao diện nhiều nhưng thiếu bằng chứng dữ liệu phía sau.

D. Hotspot và alert engine

Hotspot hiện tại được thiết kế theo hai hướng. Hướng thứ nhất là DBSCAN [13] trên không gian (`lat`, `lon`, `congestion_ratio`) để phát hiện cụm ùn tắc. Hướng thứ hai là predicted hotspot, dùng output forecast và baseline để gắn nhãn rủi ro. Ví dụ, rule có thể đánh dấu HIGH khi predicted speed 15 phút thấp hơn `p15_baseline` một ngưỡng nhất định. Trong production, các rule này cần được calibration bằng ground-truth incident/congestion labels và đánh giá precision/recall/F1.

VIII. ĐÁNH GIÁ HỆ THỐNG

A. Đánh giá kỹ thuật dữ liệu

Về kỹ thuật dữ liệu, kiến trúc có điểm mạnh ở việc tách rõ message ingestion, stream/batch processing, lakehouse storage, SQL analytics, serving API và dashboard. Các cơ chế schema contract, dead-letter queue, checkpoint/replay, lineage, partition strategy và data-quality checks đã được đặc tả ở mức thiết kế. Tuy nhiên, bằng chứng hiện tại chưa đủ để xem chúng là năng lực vận hành production: cần log producer/consumer, Spark checkpoint, Iceberg commit, Airflow DAG run, data freshness report và thống kê DLQ rate trong demo.

B. Đánh giá kỹ thuật AI

Về kỹ thuật AI, notebook đã có các thành phần quan trọng của một benchmark đáng tin cậy: leakage guard, chronological split, baseline đơn giản, nhiều model candidate, tiêu chí chọn mô hình theo triển khai, slice/error analysis, artifact packaging và smoke test inference. Điểm cần bổ sung là backtesting dài hơn, ablation study thật, kiểm tra feature parity giữa offline và online, calibration cho reliability score và ground-truth để đánh giá alert/hotspot.

IX. TÁI LẬP VÀ BẰNG CHỨNG DEMO

A. Nguyên tắc trình bày bằng chứng

Báo cáo phân nhóm artifact theo mức kiểm chứng để tránh nhầm lẫn giữa benchmark offline và hệ thống production. Nhóm *Completed* chỉ dành cho dataset, notebook hoặc output đã có thể truy cập/đối chiếu. Nhóm

Expected from notebook dành cho artifact thường được notebook sinh ra nhưng vẫn cần nộp kèm thư mục/log cụ thể khi demo. Nhóm *Required future demo evidence* là các bằng chứng bắt buộc trước khi tuyên bố pipeline/serving chạy end-to-end.

Bộ artifact nên được nộp kèm theo cấu trúc thư mục thay vì trải thành bảng checklist trong paper. Nhóm đã hoàn thành gồm Kaggle dataset, notebook và các bảng metric. Nhóm cần kèm file cụ thể gồm `model_manifest.json`, `feature_schema_used.json`, `model_joblib`, benchmark output, SHAP output và log smoke test inference. Nhóm chỉ được xem là bằng chứng hệ thống khi có demo thật gồm sample FastAPI response, dashboard screenshot, Kafka/Spark/Airflow logs và ảnh/config S3-Iceberg table layout.

B. Bộ bằng chứng tối thiểu cho demo

Với phạm vi môn học, bộ bằng chứng tối thiểu nên gồm notebook Kaggle, dataset link, model artifact, manifest/schema, benchmark output, smoke test inference, SHAP output, một sample response của FastAPI và ít nhất một screenshot dashboard. Nếu Kafka/Spark/Airflow/Iceberg chưa chạy được end-to-end, báo cáo chỉ nên ghi là *required future demo evidence* và không dùng ngôn ngữ như “đã vận hành” hoặc “production-ready”.

X. MỨC SẴN SÀNG TRIỂN KHAI

A. Yêu cầu phi chức năng

Một phân tích giao thông platform không chỉ cần dự báo tốt mà còn cần đáp ứng độ trễ, độ ổn định và khả năng quan sát. Đối với demo, mục tiêu hợp lý là dashboard cập nhật dưới 60 giây, API p95 dưới 500 ms với tải vừa phải, và pipeline có log/metric rõ khi Kafka lag hoặc Spark job fail. Với production thật, cần bổ sung autoscaling, retry policy, backpressure handling, access control, secret management và disaster recovery.

Các mục tiêu phi chức năng được đặt ở mức demo có thể kiểm chứng: dashboard cập nhật dưới 60 giây, API p95 dưới 500 ms với tải vừa phải, pipeline có Kafka lag/Spark duration/row count/task status, model có MAE/RMSE rolling theo ngày và city, S3 có lifecycle cho temporary/checkpoints sau 7–14 ngày, và secret không bị hardcoded trong code hoặc notebook. Những mục tiêu này không thay thế kiểm thử production, nhưng đủ để biến báo cáo từ mô tả kiến trúc sang bản mẫu có tiêu chí đo được.

B. Rủi ro vận hành

Các rủi ro chính gồm API quota từ nhà cung cấp, thay đổi schema dữ liệu ngoài, mất kết nối Kafka/S3, lệch múi giờ, lỗi mapping OSM và model trôi dữ liệu. Thiết kế hiện tại giảm rủi ro bằng Schema Registry, Dead Letter Queue, replay từ vùng dữ liệu gốc, data quality DAG và manifest/schema artifact cho model. Tuy nhiên, để đạt production, các cơ chế này cần được kiểm thử bằng chaos test và load test thay vì chỉ mô tả trong kế hoạch.

Bảng VIII
ACCEPTANCE GATES CHO DEMO SẢN PHẨM

Gate	Ngưỡng demo	Hành động khi fail
Data freshness	< 60 giây	Hiển thị stale badge, không ghi đè dữ liệu mới.
API latency	p95 < 500 ms	Bật cache, limit query hoặc profile endpoint.
Schema compatibility	100% pass	Chặn deploy hoặc tăng version schema.
DLQ rate	< 1% batch	Replay từ Bronze sau khi sửa rule.
Model drift	Cảnh báo theo ngày	Tạo retraining ticket, không auto-deploy.
Secret scan	0 hardcoded secret	Rotate key và fail CI.

XI. THẢO LUẬN

A. Vì sao không chọn model có MAE nhỏ nhất tuyệt đối?

Trong bảng xếp hạng 15 phút, CatBoost có MAE thấp hơn LightGBM khoảng 0.017 km/h, tương đương mức chênh rất nhỏ. Với hệ thống production-oriented, lựa chọn mô hình cần cân bằng độ chính xác, độ trễ, kích thước artifact, độ dễ triển khai, explainability và chi phí bảo trì. LightGBM có lợi thế mạnh ở tốc độ, tích hợp SHAP, tính phổ biến trong dạng bảng ML và khả năng triển khai đơn giản trong FastAPI. Vì vậy, quyết định chọn LightGBM là hợp lý khi mục tiêu là xây dựng phân tích giao thông platform chứ không chỉ tối ưu một metric offline.

B. Ý nghĩa của sai số theo km/h

MAE khoảng 1.22–1.30 km/h là mức rất thấp đối với một dataset offline đã được chuẩn hóa. Tuy nhiên, không nên diễn giải con số này như cam kết production tuyệt đối. Trong môi trường thực, lỗi API, snapshot bị thiếu, thay đổi routing, công trình hoặc sự kiện bất thường có thể làm sai số tăng. Vì vậy, báo cáo nên trình bày metric này như kết quả đánh giá benchmark trên training pack, đồng thời yêu cầu backtesting liên tục khi có data streaming thật.

C. So sánh offline đánh giá benchmark và hệ thống production

Offline notebook kiểm soát dữ liệu tốt hơn production vì tất cả record đã có sẵn và có thể làm sạch trước khi train. Production serving phải xử lý partial feature, dữ liệu đến muộn, API outage và trôi dữ liệu. Vì vậy, một mô hình tốt trong notebook chỉ là điều kiện cần. Điều kiện đủ là feature pipeline online phải sinh đúng schema như training, model manifest phải được version hóa, và dashboard phải hiển thị độ tin cậy/cập nhật cuối cùng của dự báo.

D. Threats to Validity

1) *External validity*: Dataset chỉ bao phủ 30 ngày, 20 đoạn đường và 2 thành phố, nên chưa đại diện cho toàn bộ

mạng đường Việt Nam, các mùa trong năm, ngày lễ lớn, công trình kéo dài, tai nạn nghiêm trọng hoặc khu vực ngoại thành. Vì vậy, MAE thấp trên training pack không nên được khái quát thành hiệu năng ổn định trên dữ liệu đô thị thật dài hạn.

2) *Internal validity*: Benchmark offline phụ thuộc vào feature có sẵn trong training pack và cách tạo target bằng group shift. Nếu feature pipeline online sinh schema khác, xử lý missing data khác hoặc dùng time bucket không đồng bộ với notebook, metric khi phục vụ thật có thể lệch đáng kể. Chronological split giúp giảm rủi ro leakage so với random split, nhưng chưa thay thế được backtesting dài hạn và kiểm tra feature parity offline–online.

3) *Construct validity*: MAE/RMSE đo sai số dự báo tốc độ, nhưng chưa đủ để đánh giá chất lượng alert hoặc hotspot. Hiện chưa có ground-truth label cho ùn tắc nghiêm trọng, tai nạn hoặc cảnh báo vận hành; do đó chưa thể báo cáo precision, recall và F1 cho alert engine. Các reliability/confidence score cần được calibration bằng dữ liệu thực thay vì chỉ dựa trên heuristic.

4) *Operational validity*: Production feed có thể gặp late arrival, API quota, outage, schema drift, message trùng lặp, lệch múi giờ, OSM mapping sai hoặc feature skew giữa training và serving. Các cơ chế Schema Registry, checkpoint, dead-letter queue, replay từ Bronze, lineage và monitoring đã được đặc tả, nhưng vẫn cần log end-to-end, load test và chaos test trước khi tuyên bố production-ready.

E. Lộ trình

Lộ trình sau báo cáo nên đi theo thứ tự kiểm chứng, không mở rộng công nghệ quá sớm. Giai đoạn gần nhất là hoàn thiện OSM importer, TomTom Stats pipeline và báo cáo data quality để tăng độ tin cậy của Silver/Gold. Sau đó cần đồng bộ training dataset, prediction table, MLflow registry và feature schema giữa notebook và serving. Khi dữ liệu ổn định hơn, nhóm mới nên triển khai DBSCAN hotspots, SHAP endpoint, Neo4j graph analytics và transfer learning. Giai đoạn cuối là kiểm thử E2E/load cho FastAPI, Superset/Grafana, frontend polling và các trang giám sát/cài đặt.

XII. KẾT LUẬN

Bài báo đã trình bày *Cognitive Traffic Analytics Platform*, một bản mẫu sản phẩm dữ liệu kết hợp lakehouse dữ liệu lớn và machine learning cho bối cảnh giao thông đô thị Việt Nam. Về kỹ thuật dữ liệu, hệ thống được thiết kế theo kiến trúc Kafka–Spark–S3–Iceberg–Trino–Airflow, tổ chức dữ liệu thành vùng gốc, vùng chuẩn hóa và vùng phục vụ phân tích/mô hình. Về kỹ thuật AI, notebook đã xây dựng quy trình benchmark có leakage guard, chronological split, baseline, nhiều model candidate và artifact phục vụ inference. LightGBM được chọn làm mô hình triển khai vì đạt sai số gần best model đồng thời phù hợp với yêu cầu inference nhanh, artifact nhỏ và giải thích bằng SHAP.

Kết quả thực nghiệm cho thấy hướng tiếp cận khả thi cho một bản mẫu traffic intelligence end-to-end. Tuy nhiên, báo cáo không xem kết quả offline là bằng chứng production hoàn chỉnh. Để đạt mức production-ready, hệ thống cần mở rộng coverage dữ liệu, chạy pipeline streaming/lakehouse end-to-end, tự động hóa data quality, hoàn thiện monitoring, bật model lifecycle bằng MLflow, kiểm thử API/dashboard dưới tải và xây dựng ground truth cho hotspot/alert evaluation.

LỜI CẢM ƠN

Nhóm tác giả xin cảm ơn giảng viên và trợ giảng môn Kỹ thuật và Công nghệ Dữ liệu lớn đã góp ý trong quá trình thực hiện dự án.

TÀI LIỆU

- [1] M. Zaharia *et al.*, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [2] Apache Software Foundation, “Apache Kafka Documentation,” 2026. [Online]. Available: <https://kafka.apache.org/documentation/>. Accessed: Jun. 14, 2026.
- [3] Apache Software Foundation, “Apache Iceberg Documentation,” 2026. [Online]. Available: <https://iceberg.apache.org/>. Accessed: Jun. 14, 2026.
- [4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems*, vol. 30, pp. 3149–3157, 2017.
- [5] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: Unbiased Boosting with Categorical Features,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [6] Z. Yuan and X. Li, “A Survey of Traffic Prediction: From Spatio-Temporal Data to Intelligent Transportation,” *Data Science and Engineering*, vol. 6, pp. 63–85, 2021.
- [7] Y. Li, R. Yu, C. Shahabi, and Y. Liu, “Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting,” in *Proc. Int. Conf. Learning Representations*, 2018.
- [8] B. Yu, H. Yin, and Z. Zhu, “Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting,” in *Proc. 27th Int. Joint Conf. Artificial Intelligence*, 2018, pp. 3634–3640.
- [9] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, “Graph WaveNet for Deep Spatial-Temporal Graph Modeling,” in *Proc. 28th Int. Joint Conf. Artificial Intelligence*, 2019, pp. 1907–1913.
- [10] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, “T-GCN: A Temporal Graph Convolutional Network for Traffic Prediction,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.
- [11] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [12] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [13] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining*, 1996, pp. 226–231.
- [14] V. V. Intek, “Vietnamese Kaggle Dataset,” Kaggle Dataset, 2026. [Online]. Available: <https://www.kaggle.com/datasets/vv1ntek/vietnamese-kaggle-dataset/>. Accessed: Jun. 14, 2026.
- [15] V. V. Intek, “Traffic Forecasting Benchmark Notebook,” Kaggle Notebook, 2026. [Online]. Available: <https://www.kaggle.com/code/vv1ntek/notebookfe1af77eb9>. Accessed: Jun. 14, 2026.